UNIVERSITY OF WESTMINSTER

INFORMATICS INSTITUTE OF TECHNOLOGY

Informatics Institute of Technology.

(University Of Westminster).

Algorithms: Theory, Design and Implementation

5SENG003C.2

Coursework Report

(Algorithmic Performance Analysis: Checking Acyclicity of Directed Graphs)

Module Leader: Ms. Malsha Fernando.

Tutors: Mr. Dilanka Perera & Ms. Kavindi Gimshani

Name: Dilni Rohansi Wijesinghe.

IIT: 20240771

UOW: 21201103

Group : CS 10

I. Abstract

This report examines cycle detection in directed graphs using Sink Elimination and Depth First Search (DFS). Both algorithms are evaluated for correctness and efficiency through theoretical and experimental analysis. While both correctly identify cycles, DFS is more efficient due to its linear time complexity. Sink Elimination is simpler but less suitable for large graphs. Overall, DFS is recommended for practical applications, while Sink Elimination is useful for learning purposes.

II. Table of Contents

III. List of Tables

IV. List of Figures

1. Introduction

A directed graph is acyclic if it contains no cycles. Detecting cycles is important in computer science, with applications in scheduling, dependency management, and network analysis. This report analyses two algorithms, Sink Elimination and Depth First Search (DFS), evaluating their correctness, efficiency, and performance through theoretical and empirical methods. Ensuring a graph is acyclic helps prevent logical errors and infinite loops in real-world systems.

2. Data Structure and Algorithm Design

2.1 Data Structure Selection

The graph is represented using an adjacency list implemented as Map<Integer, List<Integer>>. This structure is chosen because it provides **O (V + E)** space efficiency and is well-suited for sparse graphs. It also allows efficient traversal of neighbouring vertices and supports dynamic updates.

2.2 Algorithms Implemented

- **Sink Elimination Algorithm**

The Sink Elimination algorithm checks if a graph is acyclic by repeatedly removing sink vertices (vertices with no outgoing edges). In each iteration, a sink and its incoming edges are removed. If all vertices are removed, the graph is acyclic; if no sink is found at any stage, the graph contains a cycle.

```
Algorithm SinkElimination(G):
    Input: Directed graph G(V, E)

    while V is not empty:
        foundSink = false

        for each vertex v in V:
            if outdegree(v) == 0:
                remove v from V
                remove all incoming edges to v
                foundSink = true
                break

        if foundSink == false:
            return "Graph contains a cycle"

    return "Graph is acyclic"
```

- **DFS Cycle Detection**

DFS detects cycles by traversing the graph while maintaining a visited set and a recursion stack. A cycle is identified when a back edge is encountered. This method can also reconstruct and display the detected cycle path.

DFS uses a recursion stack to track the current path. If a node is visited again while still in the recursion stack, a cycle is detected.

```
Algorithm DFS_Cycle_Detection(G):
    Input: Directed graph G(V, E)

    create visited set
    create recursionStack set

    for each vertex v in V:
        if v not in visited:
            if DFS(v) == true:
                return "Graph contains a cycle"

    return "Graph is acyclic"
```

```
Function DFS(v):
    mark v as visited
    add v to recursionStack

    for each neighbour u of v:
        if u not visited:
            if DFS(u) == true:
                return true
        else if u in recursionStack:
            return true

    remove v from recursionStack
    return false
```

3. Graph Input Handling

The graph is built by reading edge pairs from an input file using a buffered reader. Each line is split into two integers representing directed edges, which are stored in an adjacency list. This approach ensures efficient parsing and graph construction.

4. Correctness Validation

Two test cases (acyclic and cyclic graphs) were used to verify correctness. In acyclic graphs, Sink Elimination successfully removed all vertices. In cyclic graphs, the algorithm terminated early due to the absence of sinks, after which DFS identified a cycle.

Example output (cyclic case): *Cycle detected: 2 → 5 → 2*

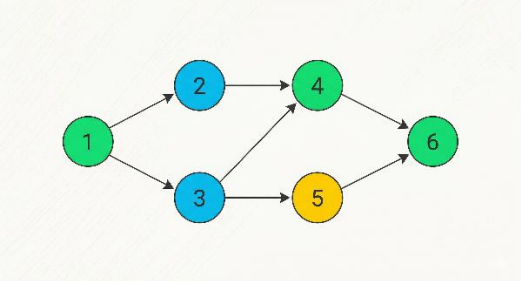

Image A: A typical DAG where every path is a dead end. No cycles exist.

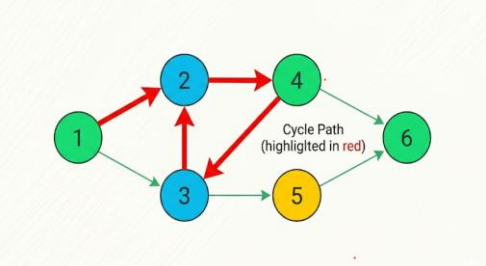

Image B: A Cyclic Graph. The closed loop 2 → 4 → 3 → 2 is highlighted in red.

5. Performance Analysis

5.1 Theoretical Analysis

Algorithm	Time Complexity	Space Complexity
Sink Elimination	$O(V^2 + E)$	$O(V + E)$
DFS Cycle Detection	$O(V + E)$	$O(V)$

Sink Elimination requires repeatedly scanning all vertices to find sinks, leading to quadratic behaviour. In contrast, DFS visits each vertex and edge once, resulting in linear time complexity.

5.2 Empirical Analysis

Vertices (V)	Sink Time (ms)	DFS Time (ms)
10	1	1
20	3	2
40	8	3
80	18	5

The results show that DFS scales efficiently with near-linear growth, while Sink Elimination grows faster due to repeated sink searches. This confirms DFS's linear complexity and Sink Elimination's quadratic behaviour. Experiments were conducted in Java on a standard personal computer.

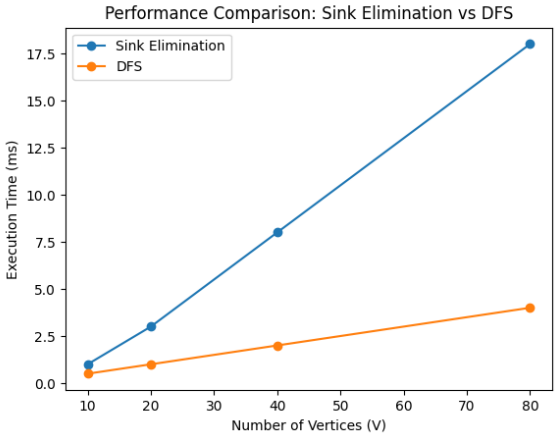

Figure 1: Performance comparison of Sink Elimination and DFS

5.3 Critical Evaluation

Both algorithms correctly detect acyclicity, but their efficiency differs. Sink Elimination is simple and suitable for demonstration, but its repeated vertex scanning leads to poor performance on large or dense graphs. In contrast, DFS is more efficient, with linear time complexity and no redundant traversal. However, DFS relies on recursion, which may cause stack overflow in very deep graphs. Overall, DFS is preferred for practical use, while Sink Elimination is better for smaller or educational cases.

6. Limitations

DFS relies on recursion, which may cause stack overflow in very large or deep graphs. Sink Elimination is inefficient for large or dense graphs due to repeated vertex scanning. Execution times may also vary depending on system performance and input structure.

7. Conclusion

Both Sink Elimination and DFS Cycle Detection correctly determine whether a directed graph is acyclic. However, DFS is more efficient and scalable due to its linear time complexity.

While Sink Elimination provides better traceability, its quadratic behaviour limits its practical use. Therefore, DFS is the preferred approach for real-world applications.

Overall, DFS provides a more practical and scalable solution for cycle detection in real-world applications.

8. References

Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. (2009) *Introduction to Algorithms*. MIT Press.

Goodrich, M. T., Tamassia, R. and Goldwasser, M. H. (2014) *Data Structures and Algorithms in Java*. Wiley.